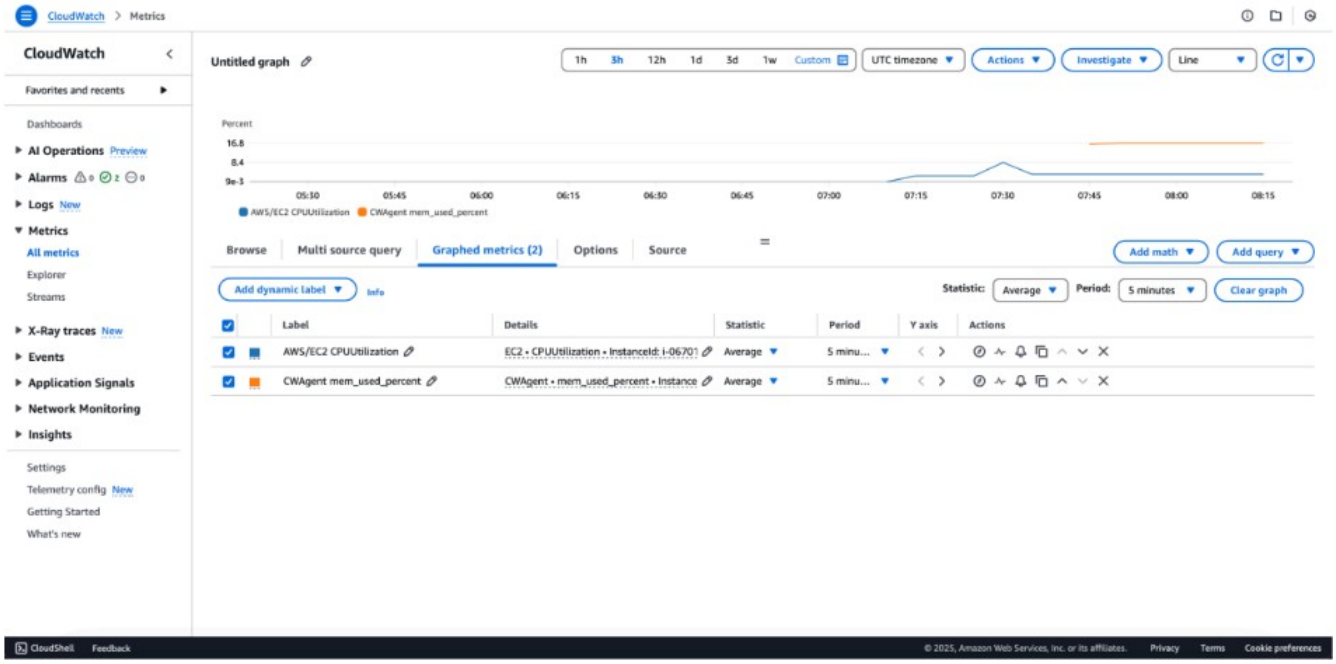
Step 1: Implementing Resource Monitoring via AWS CloudWatch

Technical Overview

In this phase, I configured comprehensive monitoring for the EC2 instance to ensure operational visibility and security baseline auditing. By utilizing **Amazon CloudWatch**, I gained real-time insights into system performance and resource consumption.

Key Metrics Monitored:

- **CPU Utilization:** Tracks the processing load on the instance. Significant spikes may indicate a potential Denial of Service (DoS) attempt or an inefficient process that needs optimization.
- **Memory Utilization (via CloudWatch Agent):** Unlike CPU, memory metrics are not collected by default at the hypervisor level. I deployed the **CloudWatch Agent** on the instance to push custom OS-level metrics, providing a deeper view of the system's health.



Step 2: Proactive Threat Detection with CloudWatch Alarms

Technical Overview

While monitoring provides visibility (as shown in Step 1), effective cloud operations require proactive detection. In this step, I transitioned from passive monitoring to automated alerting by configuring **CloudWatch Alarms** based on critical performance and security baselines.

Security Baselines and Alerts Configured:

I defined and implemented precise alert conditions to detect anomalies that might indicate operational failure or a security incident. The alarms shown in the screenshot serve distinct purposes:

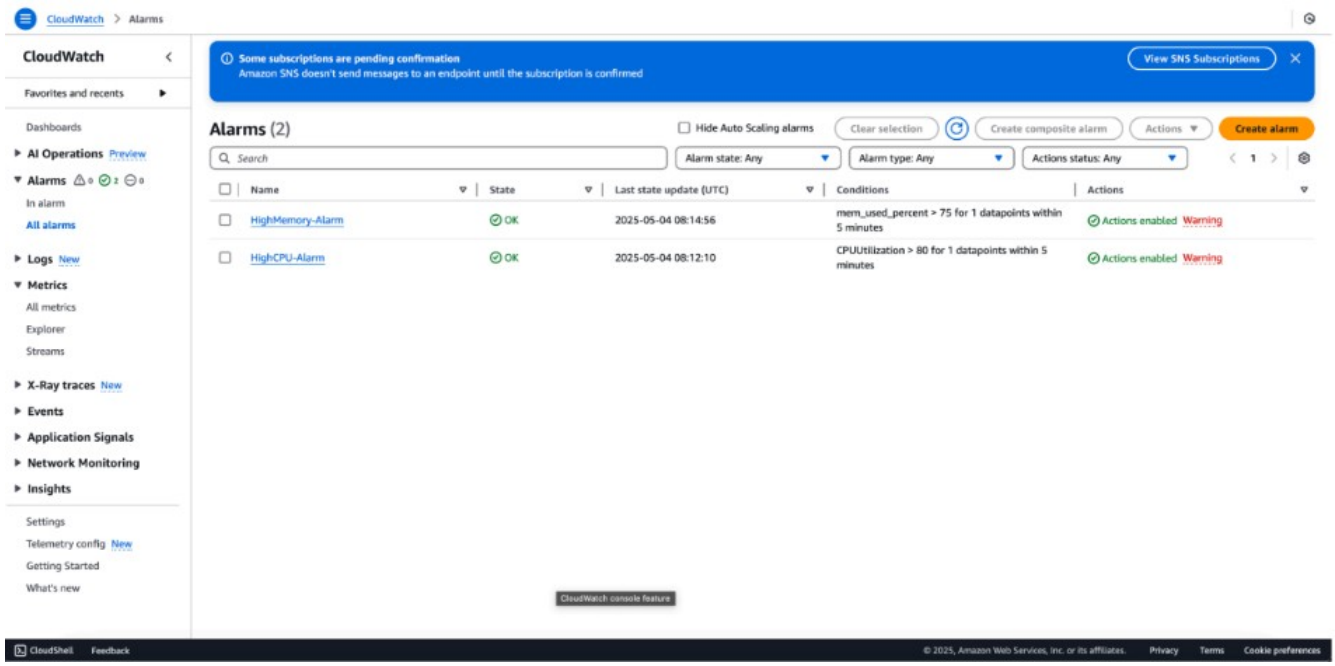
1. **High CPU Utilization Alarm:**
    - **Threshold:** > 80% sustained for 5 consecutive minutes.
    - **Rationale:** Detects sustained high-load situations. Beyond performance bottlenecks, this is a critical early warning for potential resource-exhaustion attacks, such as distributed

denial-of-service (DDoS) attempts or unauthorized cryptocurrency mining (cryptojacking) by an attacker who has compromised the instance.

2. **High Memory Utilization Alarm:**
    - **Threshold:** > 75% sustained for 5 consecutive minutes.
    - **Rationale:** Leverages the custom metric from the CloudWatch Agent. Sustained high memory usage is a critical indicator of a memory leak or a sophisticated buffer overflow attempt, allowing security teams to intervene before a system crash occurs.

Integrated Notification Layer:

These alarms are linked to an **Amazon SNS (Simple Notification Service)** topic. When an alarm state is triggered, an automated email notification is dispatched to the operational team, ensuring immediate visibility into the incident.



Step 3: Orchestrating Real-Time Incident Notifications via Amazon SNS

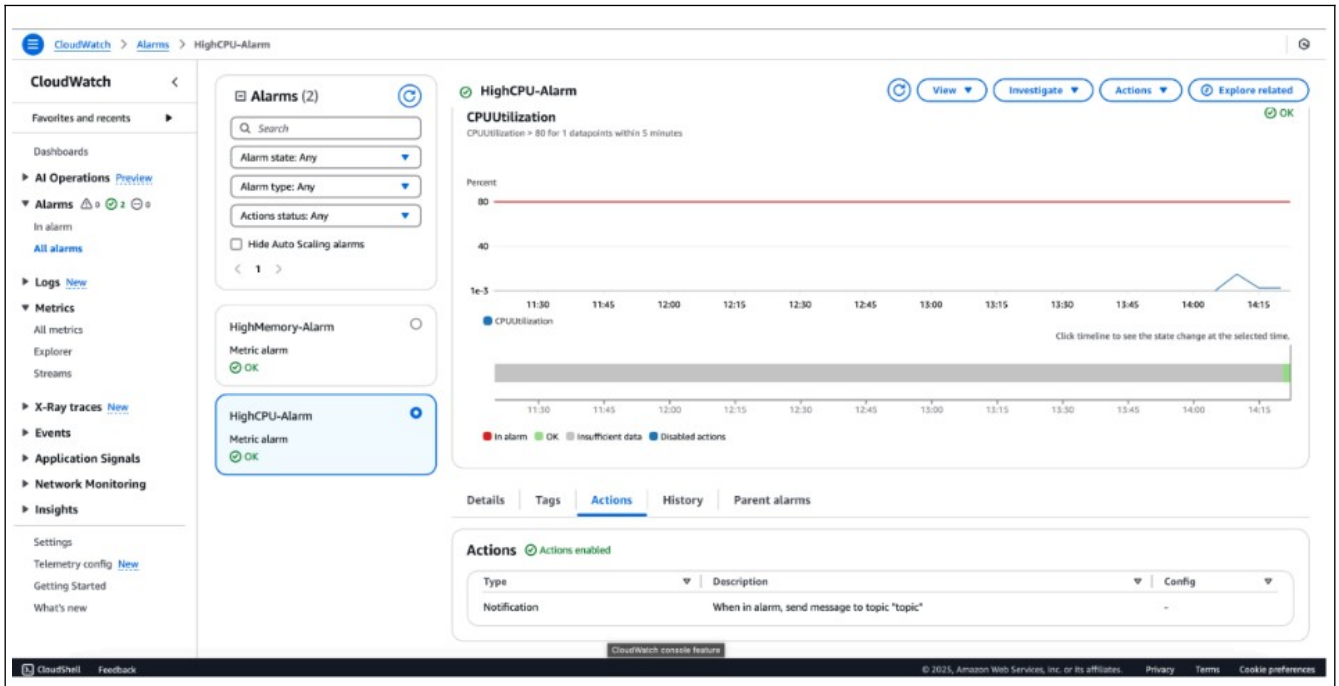
Technical Overview

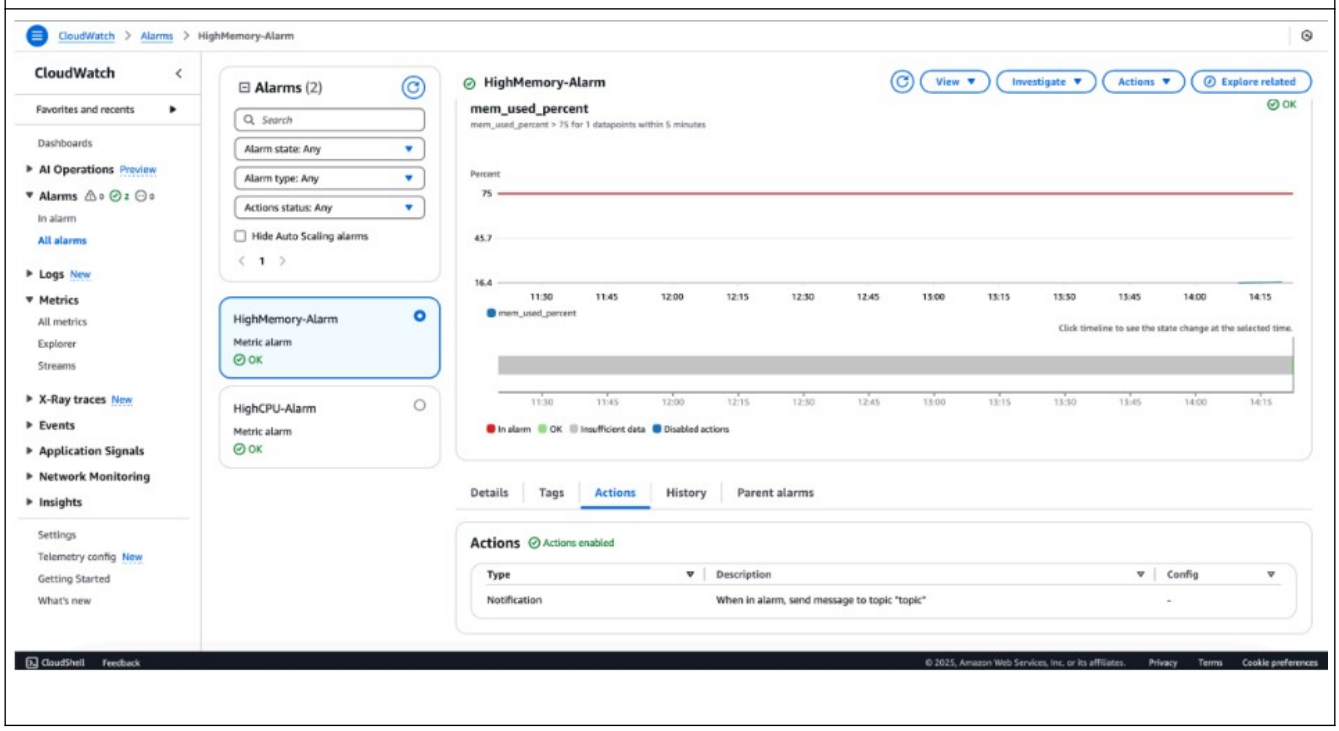
To ensure immediate awareness of potential security or performance issues, I integrated **Amazon Simple Notification Service (SNS)** with the CloudWatch Alarms configured in the previous step. This creates an automated notification pipeline that bridges the gap between detection and manual or automated intervention.

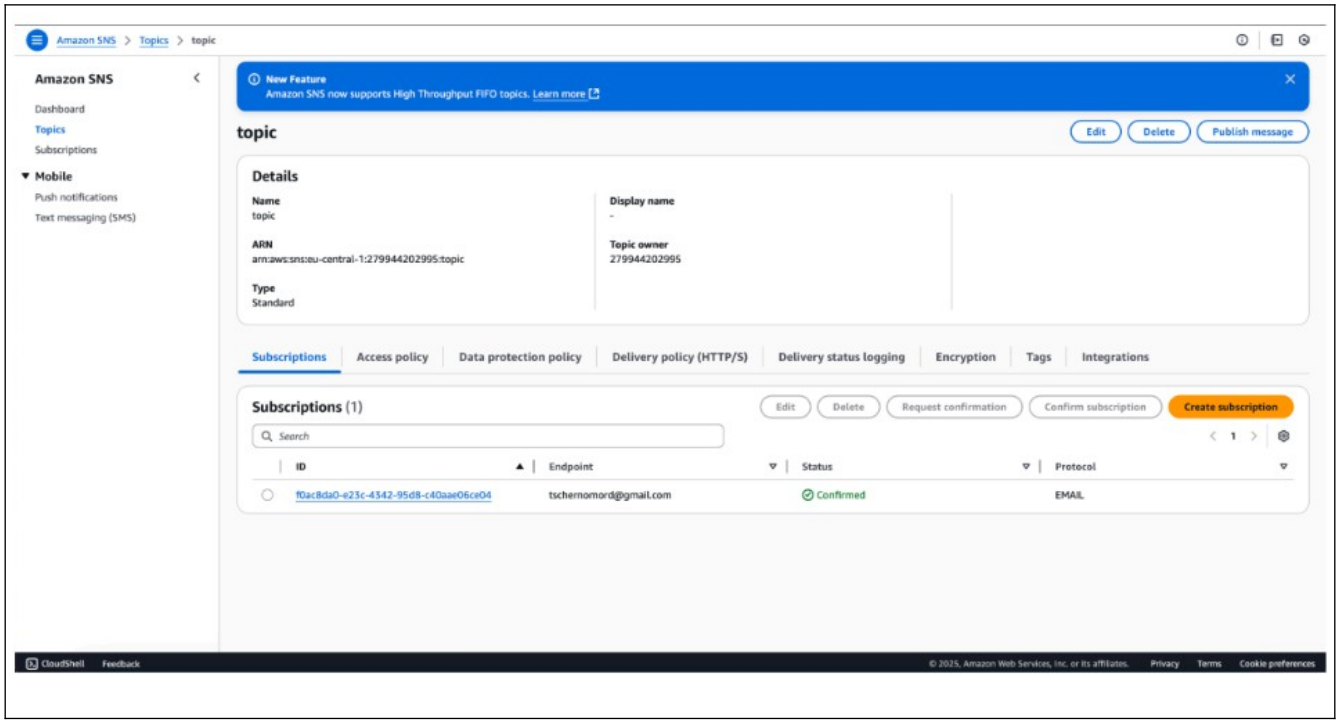
Implementation Details:

- **SNS Topic Creation:** I created a dedicated SNS topic (standard type) to serve as a centralized hub for all resource-related alerts.
- **Protocol Configuration:** Configured an **Email Protocol** subscription to ensure that critical alerts are pushed directly to the security analyst's inbox.

- **Subscription Validation:** As seen in the screenshots, the subscription status is **"Confirmed."** This is a critical step in AWS to prevent "dead-letter" notifications and ensure that the communication channel is verified and active.







Step 4: System Stress Testing and Operational Validation Technical Overview

A monitoring system is only as reliable as its ability to trigger under real-world conditions. In this final phase, I performed a **Stress Test** on the EC2 instance to manually breach the defined thresholds and validate the end-to-end alerting pipeline (from metric collection to SNS notification).

Execution Steps:

1. **SSH Access:** Established a secure terminal session to the EC2 instance.
2. **Synthetic Load Generation:** I utilized the stress utility (or a high-intensity background process) to force CPU utilization above the 80% threshold:

   Bash

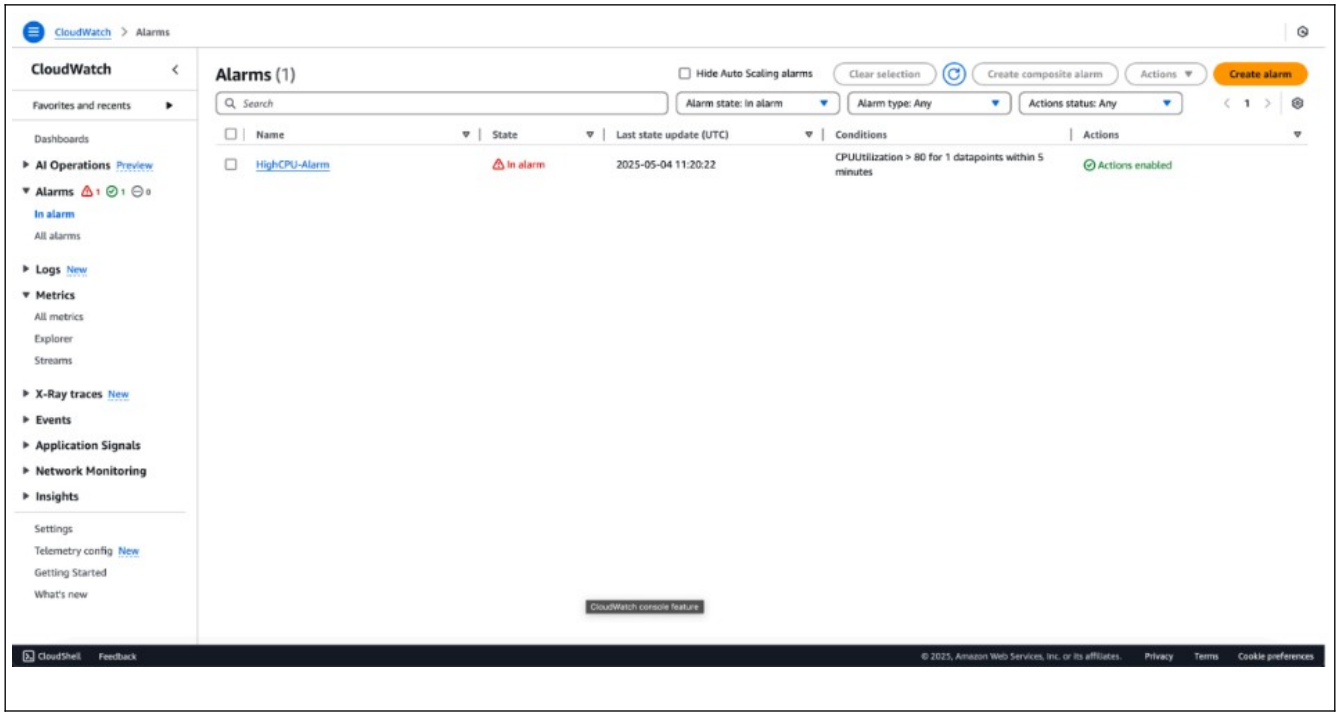
   ```
   # Generating 100% load on 2 CPU cores for 5 minutes

   stress --cpu 2 --timeout 300
   ```

3. **Observation:** Monitored the CloudWatch dashboard as the metric points were aggregated. After the 5-minute evaluation period, the state transitioned from **OK** to **In alarm**.

Validation Logic:

The alarm was triggered based on the following condition:

$$\text{Average}(CPUUtilization) > 80\% \text{ for 1 datapoint within 5 minutes}$$



Step 5: Verifying Alert Delivery and Incident Metadata

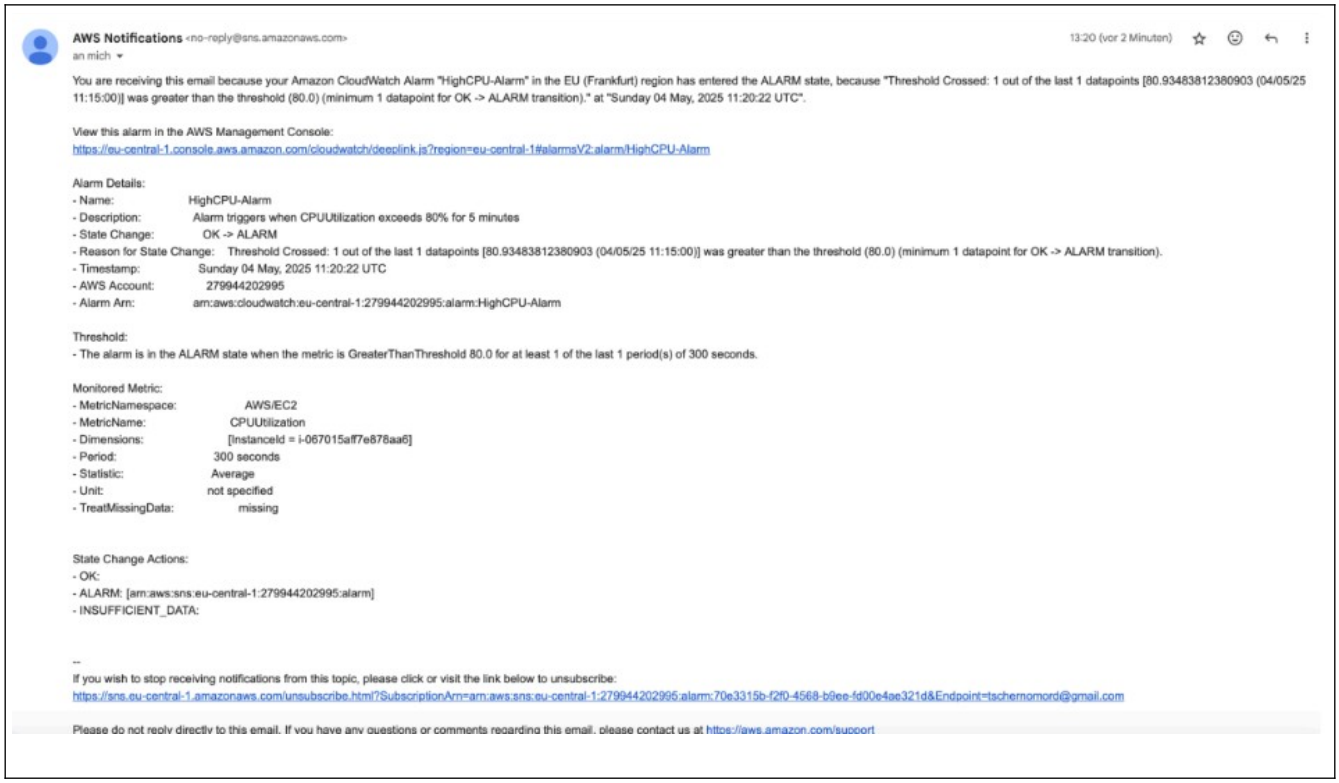
Technical Overview

The final validation of the monitoring pipeline is the successful delivery of the notification. This step confirms that the **Amazon SNS** service effectively communicated with the external email endpoint once the **CloudWatch Alarm** threshold was breached.

Incident Metadata Analysis:

The received notification provides critical data points necessary for an initial security or operational triage:

- **Status Change:** OK -> ALARM
- **Threshold Breach:** The email captures the exact value that triggered the alert ($80.93\% > 80.0\%$), providing immediate context without needing to log into the console first.
- **Resource Identification:** Includes the specific **Alarm ARN** and **Instance ID**, allowing an incident responder to quickly identify which asset is under stress.
- **Timestamp:** Provides a precise UTC record for the event, which is essential for correlation during a forensic investigation or post-mortem analysis.



AWS Notifications <no-reply@sns.amazonaws.com>
an mich

13:20 (vor 2 Minuten)

You are receiving this email because your Amazon CloudWatch Alarm "HighCPU-Alarm" in the EU (Frankfurt) region has entered the ALARM state, because "Threshold Crossed: 1 out of the last 1 datapoints [80.93483812380903 (04/05/25 11:15:00)] was greater than the threshold (80.0) (minimum 1 datapoint for OK -> ALARM transition)." at "Sunday 04 May, 2025 11:20:22 UTC".

View this alarm in the AWS Management Console:
https://eu-central-1.console.aws.amazon.com/cloudwatch/deeplink.js?region=eu-central-1#alarmsV2:alarm/HighCPU-Alarm

Alarm Details:
- Name: HighCPU-Alarm
- Description: Alarm triggers when CPUUtilization exceeds 80% for 5 minutes
- State Change: OK -> ALARM
- Reason for State Change: Threshold Crossed: 1 out of the last 1 datapoints [80.93483812380903 (04/05/25 11:15:00)] was greater than the threshold (80.0) (minimum 1 datapoint for OK -> ALARM transition).
- Timestamp: Sunday 04 May, 2025 11:20:22 UTC
- AWS Account: 279944202995
- Alarm Arn: arn:aws:cloudwatch:eu-central-1:279944202995:alarm:HighCPU-Alarm

Threshold:
- The alarm is in the ALARM state when the metric is GreaterThanThreshold 80.0 for at least 1 of the last 1 period(s) of 300 seconds.

Monitored Metric:
- MetricNamespace: AWS/EC2
- MetricName: CPUUtilization
- Dimensions: [InstanceId = i-067015aff7e878aa6]
- Period: 300 seconds
- Statistic: Average
- Unit: not specified
- TreatMissingData: missing

State Change Actions:
- OK:
- ALARM: [arn:aws:sns:eu-central-1:279944202995:alarm]
- INSUFFICIENT_DATA:

--
If you wish to stop receiving notifications from this topic, please click or visit the link below to unsubscribe:
https://sns.eu-central-1.amazonaws.com/unsubscribe.html?SubscriptionArn=arn:aws:sns:eu-central-1:279944202995:alarm:70e3315b-f2f0-4568-b9ee-fd00e4ae321d&Endpoint=tschernomord@gmail.com

Please do not reply directly to this email. If you have any questions or comments regarding this email, please contact us at https://aws.amazon.com/support

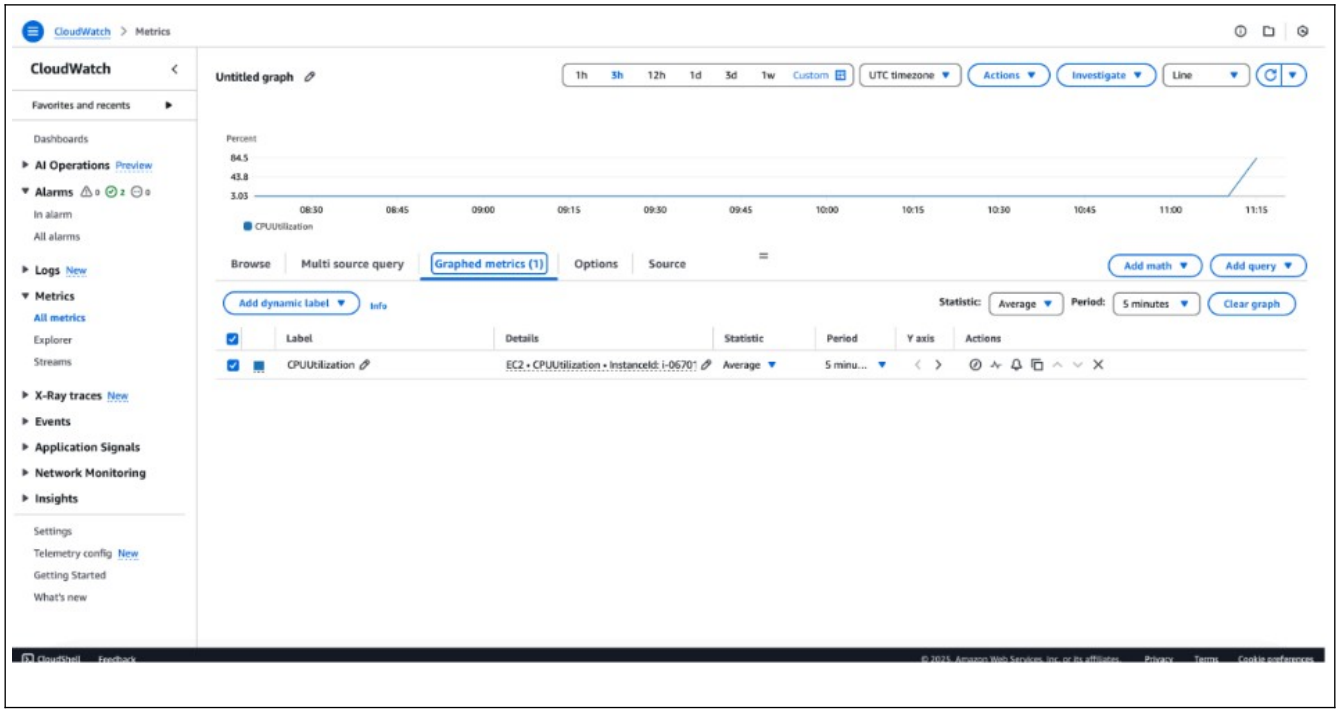
Step 6: Visualizing the Anomaly via CloudWatch Metrics

Technical Overview

Data visualization is a critical component of security monitoring. In this step, I analyzed the **CPUUtilization** metric graph to observe the physical impact of the stress test on the instance. This visualization confirms the correlation between the manual execution of the stress command and the subsequent breach of the alarm threshold.

Observations:

- **Baseline Activity:** The graph shows a near-zero baseline before the test, representing the instance in an idle state.
- **The Spike:** A sharp, vertical ascent in the metric data points indicates the exact moment the synthetic load was applied.
- **Threshold Breach:** The peak of the graph clearly exceeds the 80% horizontal threshold, providing the visual justification for the alarm state triggered in Step 4.



Step 7: Resource Decommissioning and Infrastructure Hygiene

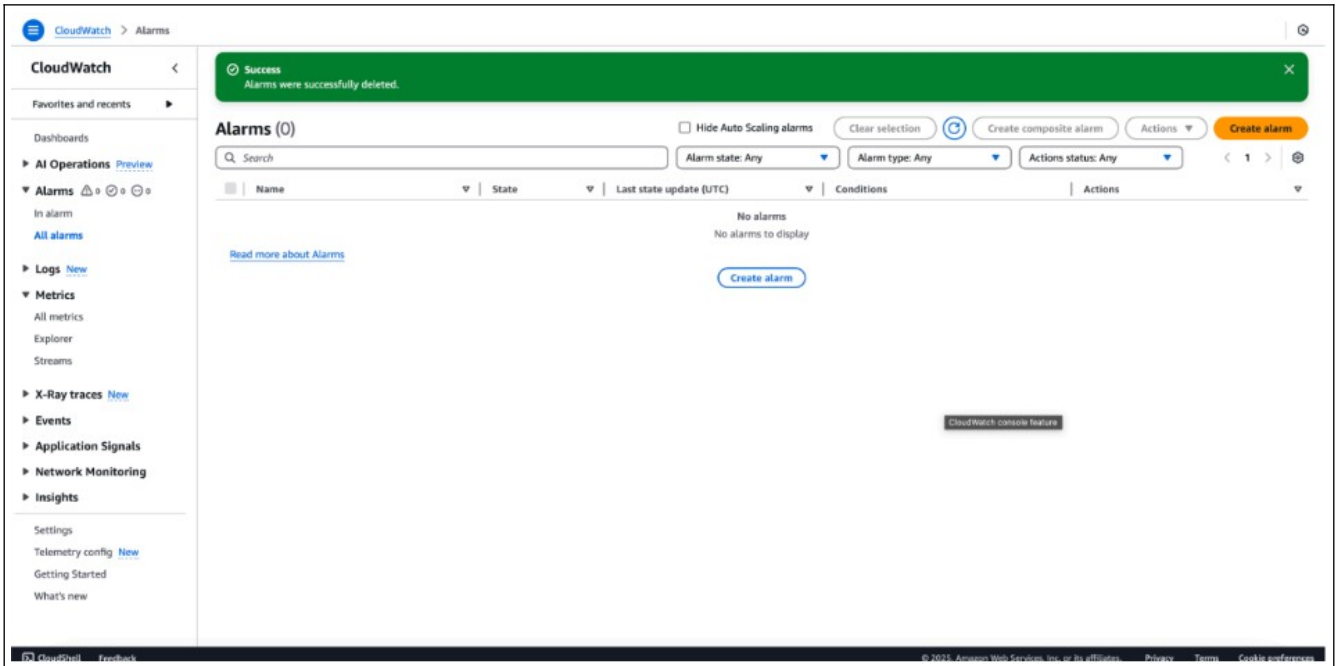
Technical Overview

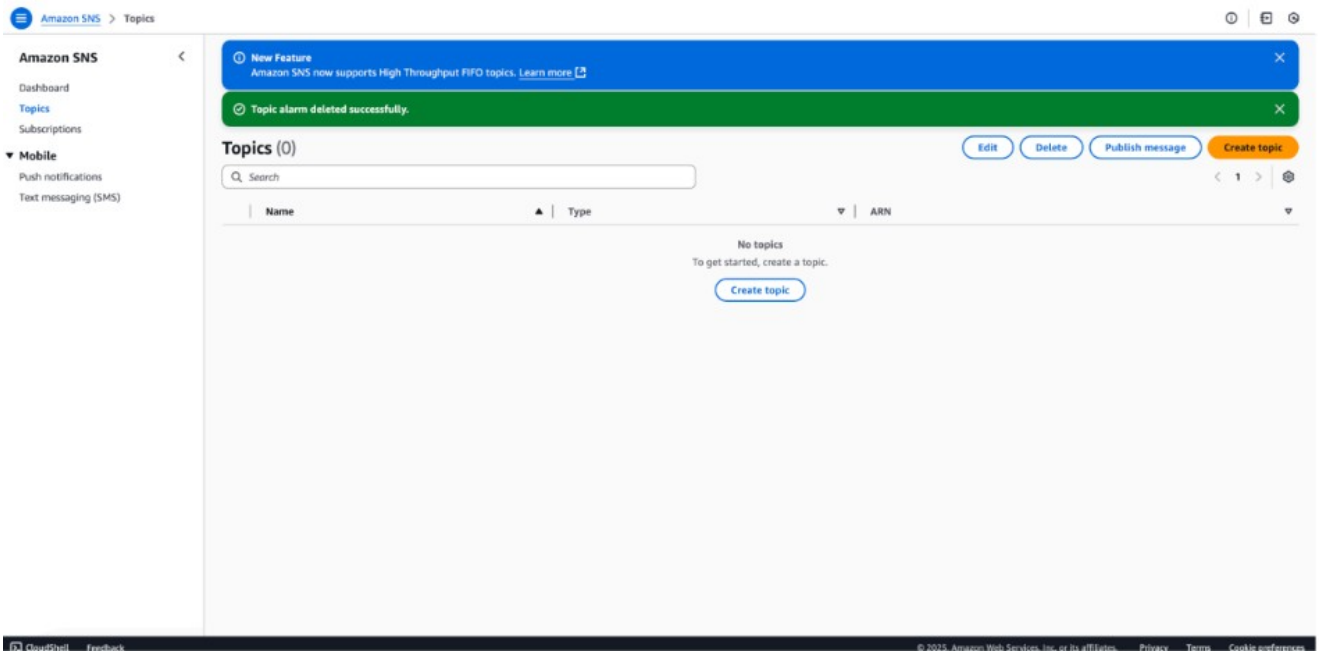
The lifecycle of cloud infrastructure includes not only deployment and monitoring but also the secure and efficient removal of resources. In this final phase, I performed a systematic cleanup of the laboratory environment to ensure cost-efficiency and prevent "infrastructure sprawl."

[Images showing successful deletion of CloudWatch Alarms and Amazon SNS Topics]

Cleanup Procedures:

1. **Alarm Decommissioning:** Deleted all custom CloudWatch Alarms to stop further metric evaluation and potential noise.
2. **Notification Pipeline Removal:** Deleted the SNS topics and associated subscriptions, ensuring that no stale communication channels remain in the AWS account.
3. **Verification:** Confirmed through the AWS Console that all experimental resources were successfully terminated, leaving the environment in a clean, baseline state.


CloudWatch > Alarms
CloudWatch
Favorites and recents
Dashboards
AI Operations Preview
Alarms
In alarm
All alarms
Logs New
Metrics
All metrics
Explorer
Streams
X-Ray traces New
Events
Application Signals
Network Monitoring
Insights
Settings
Telemetry config New
Getting Started
What's new
Success
Alarms were successfully deleted.
Alarms (0)
Hide Auto Scaling alarms
Clear selection
Create composite alarm
Actions
Create alarm
Search
Alarm state: Any
Alarm type: Any
Actions status: Any
Name
State
Last state update (UTC)
Conditions
Actions
No alarms
No alarms to display
Read more about Alarms
Create alarm
CloudWatch console feature
CloudShell
Feedback
© 2025, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Amazon SNS > Topics
Amazon SNS
Dashboard
Topics
Subscriptions
Mobile
Push notifications
Text messaging (SMS)
New Feature
Amazon SNS now supports High Throughput FIFO topics. Learn more
Topic alarm deleted successfully.
Topics (0)
Edit
Delete
Publish message
Create topic
Search
Name
Type
ARN
No topics
To get started, create a topic.
Create topic
CloudShell
Feedback
© 2025, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences